

AWS Machine Learning Basics Lab

House Price Predictor – Assignment Report

Submitted By: *Shrivathsan K M* (2026a1c200038)

Programme: BITS Pilani – Professional Certificate in AI/ML

Lab Title: House Price Predictor – AWS Basics Lab

1. Objective

The objective of this lab was to build an end-to-end machine learning workflow on AWS for predicting house prices using a synthetic dataset. The solution demonstrates the complete lifecycle of an ML project including data storage, data cataloging, querying, feature management, model training, and model registration using AWS services. This aligns with the business problem of building an auditable and version-controlled house price prediction system.

2. Business Problem

A real estate company requires an automated system capable of predicting house prices based on property characteristics. Instead of relying on manual comparisons and human intuition, the system should generate consistent and data-driven price estimates while maintaining complete traceability from raw data to the trained machine learning model.

3. Dataset

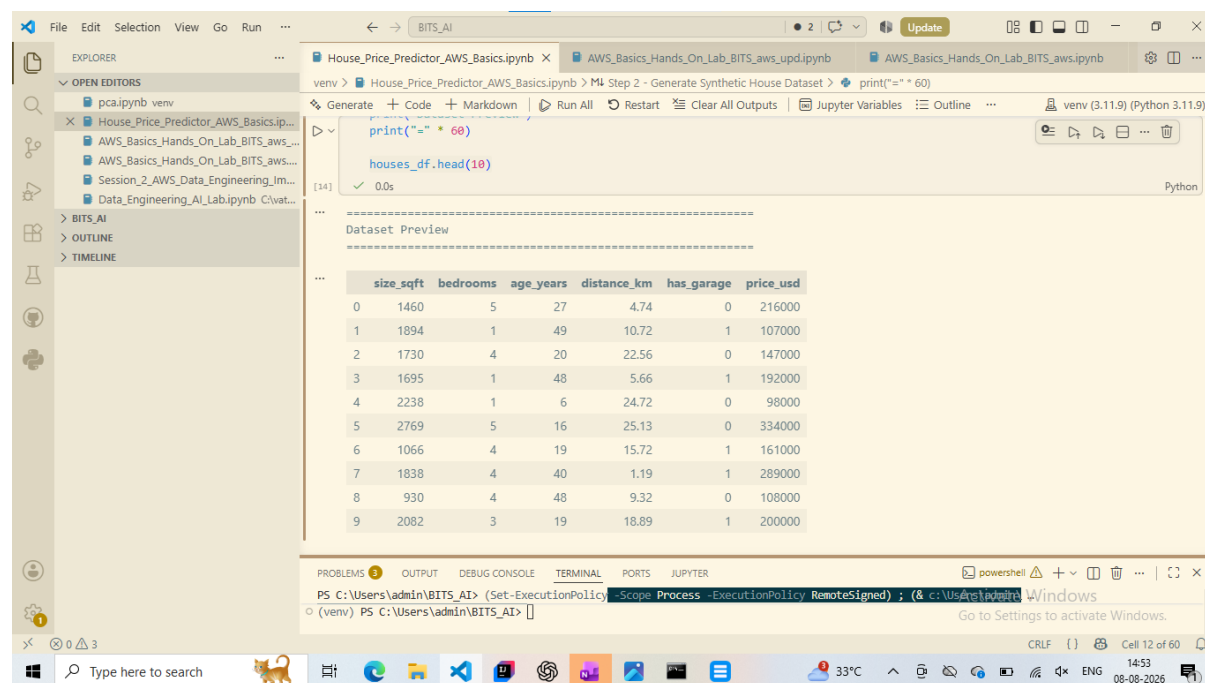
A synthetic dataset containing **500 house records** was generated using NumPy within the Jupyter notebook.

Each record contains the following attributes:

Feature	Description
size_sqft	Size of the house in square feet
bedrooms	Number of bedrooms
age_years	Age of the house
distance_km	Distance from city centre
has_garage	Indicates whether the house has a garage
price_usd	Target variable (house price)

The house prices were generated using the mathematical formula provided in the assignment along with Gaussian noise to simulate real-world market variations.

Fig 1 - Dataset Preview



4. AWS Services Used

The following AWS services were used during the implementation.

AWS Service	Purpose
Amazon S3	Store dataset, Athena results and model artifact
AWS Glue Data Catalog	Register dataset as an external table
Amazon Athena	Query the dataset using SQL
SageMaker Feature Store	Store ML features
SageMaker Model Registry	Register and version the trained model

These services were selected because they can be used without launching SageMaker training instances, making the lab lightweight and cost-effective.

5. Implementation Steps

The implementation was completed in the following sequence.

Step 1 – Environment Preparation

- Configured AWS CLI
- Configured boto3

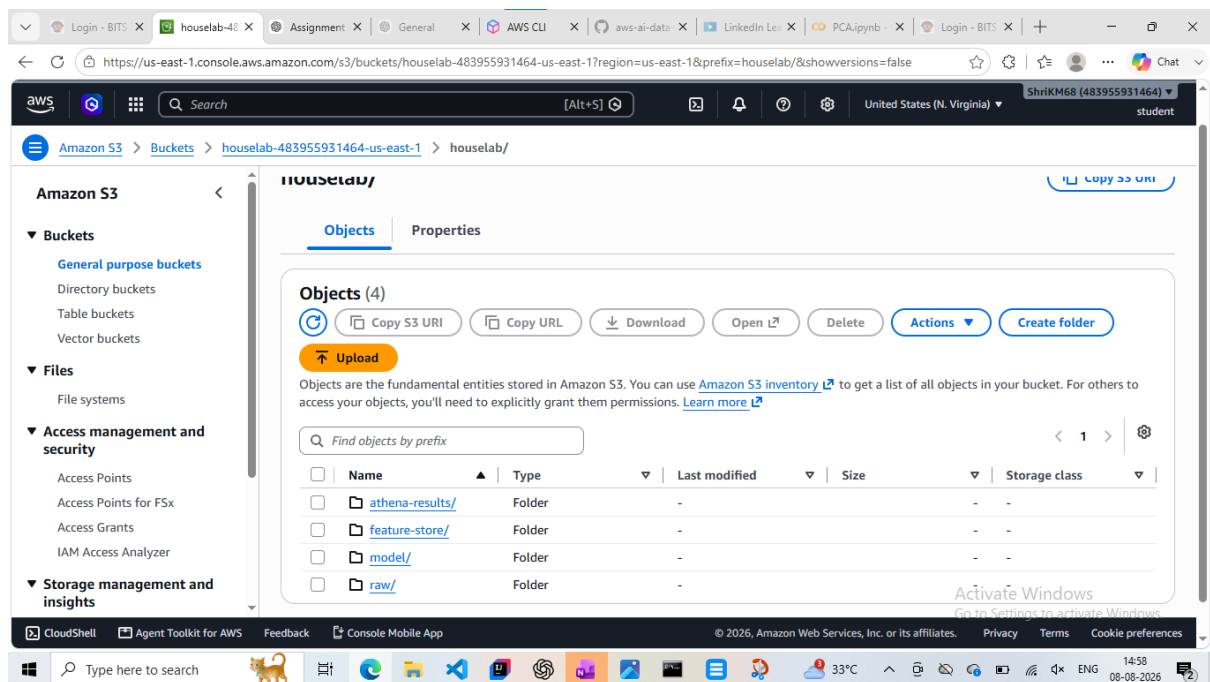
- Verified AWS credentials
- Connected to AWS using IAM user

Step 2 – Amazon S3

Created an S3 bucket to store:

- Raw dataset
- Athena query results
- Feature Store offline data
- Model artifact

Fig 2 - S3 Bucket Structure



Step 3 – Dataset Upload

Generated the synthetic dataset locally and uploaded it as:

houses.csv

inside the S3 bucket.

Verified the upload successfully.

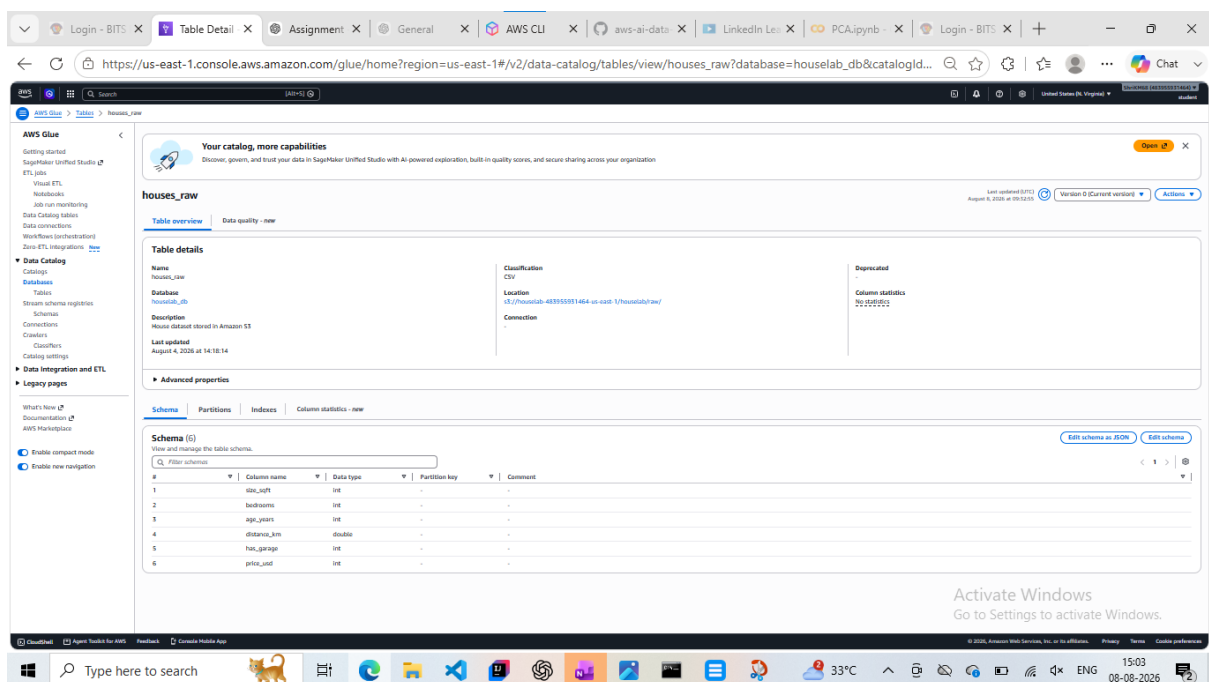
Step 4 – AWS Glue Data Catalog

Created:

- Glue Database
- External Table (houses_raw)

This enabled Athena to query the CSV file stored in Amazon S3.

Fig 3 Glue Database & Table:

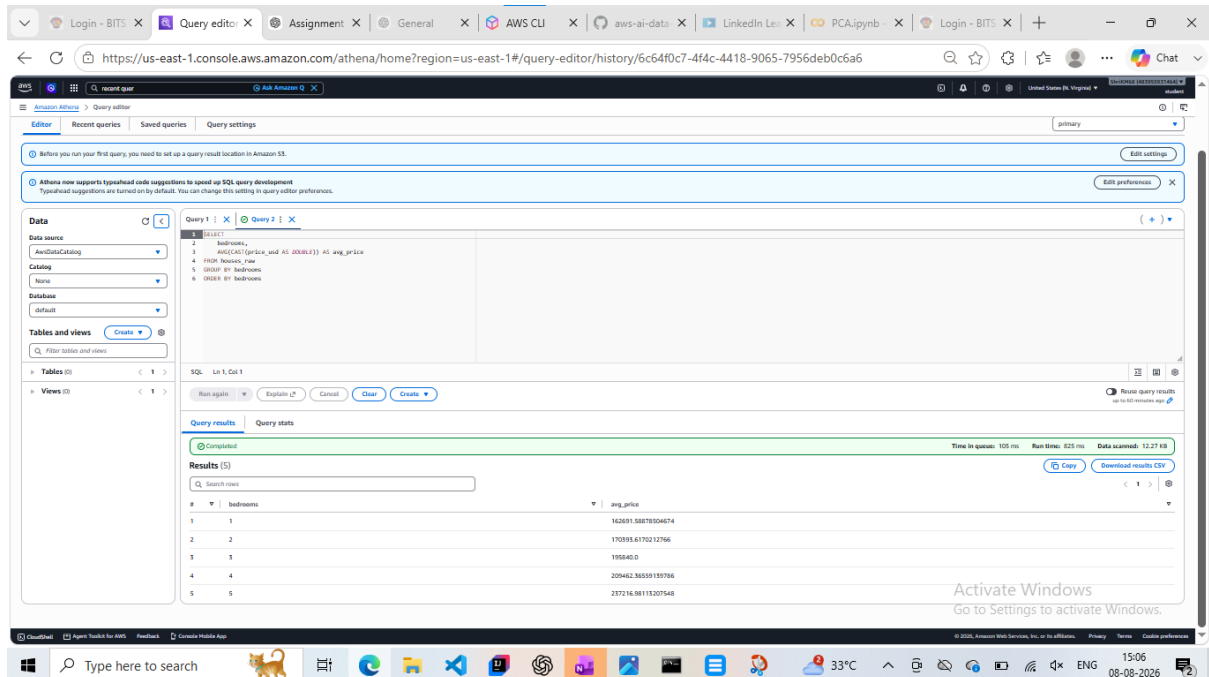


Step 5 – Amazon Athena

Executed an SQL query to calculate the average house price grouped by the number of bedrooms.

The query executed successfully and the results were stored in Amazon S3.

Fig 4 - Athena Result



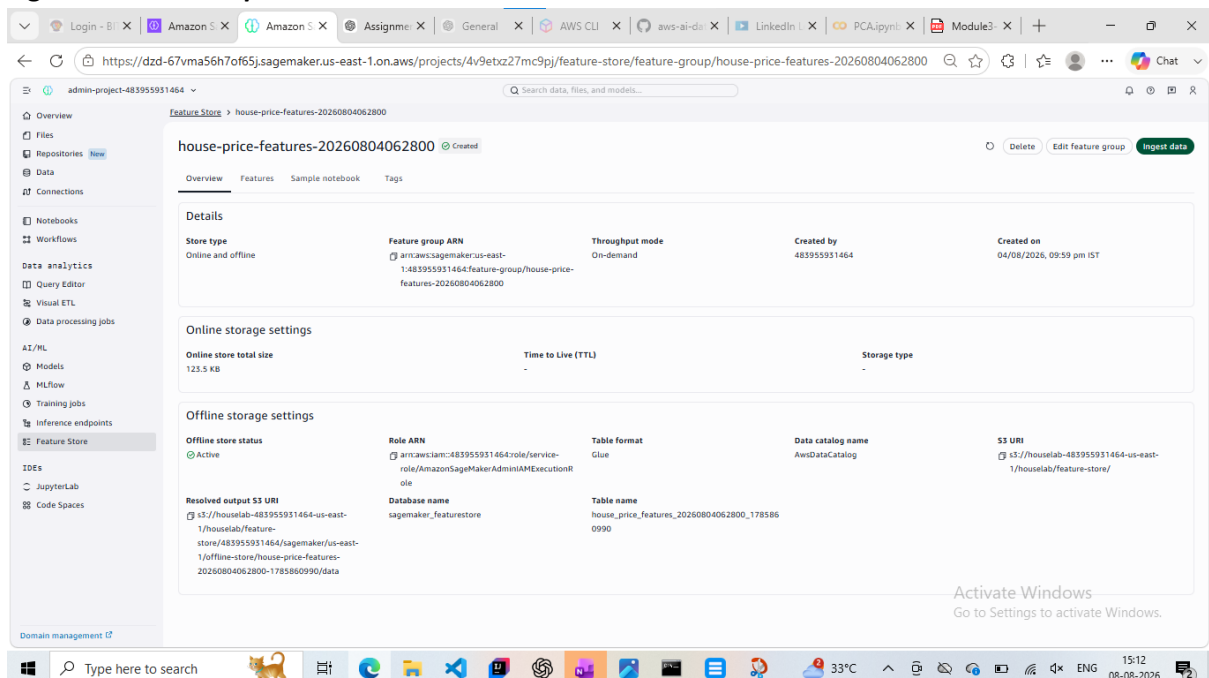
Step 6 – SageMaker Feature Store

Prepared the dataset by adding:

- record_id
- event_time

Created the Feature Group and successfully ingested all **500 records**.

Fig 5 - Feature Group



Step 7 – Model Training

A Random Forest Regressor was trained locally using Scikit-Learn.

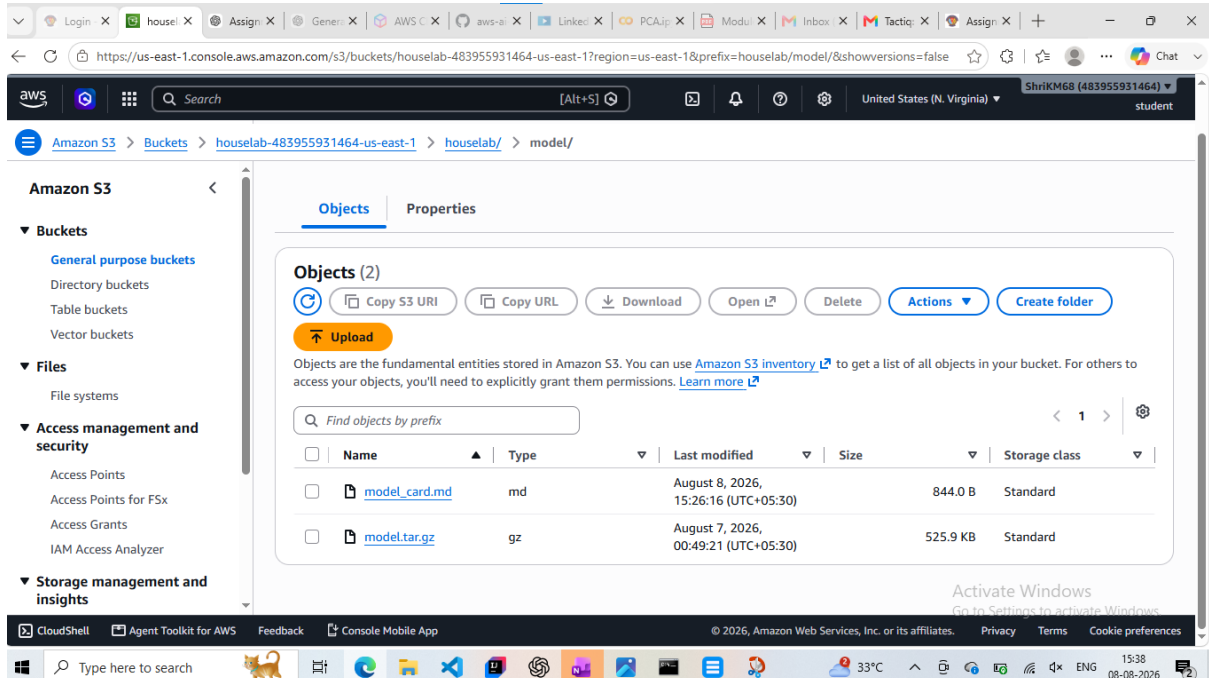
Training configuration:

Parameter	Value
Algorithm	Random Forest Regressor
Number of Trees	100
Training Split	80%
Test Split	20%
Random State	42

Step 8 – Model Packaging

The trained model was:

- Serialized using Pickle
- Packaged as `model.tar.gz`
- Uploaded **model.tar.gz** and **model_card.md** to Amazon S3 for model artifact storage and governance.

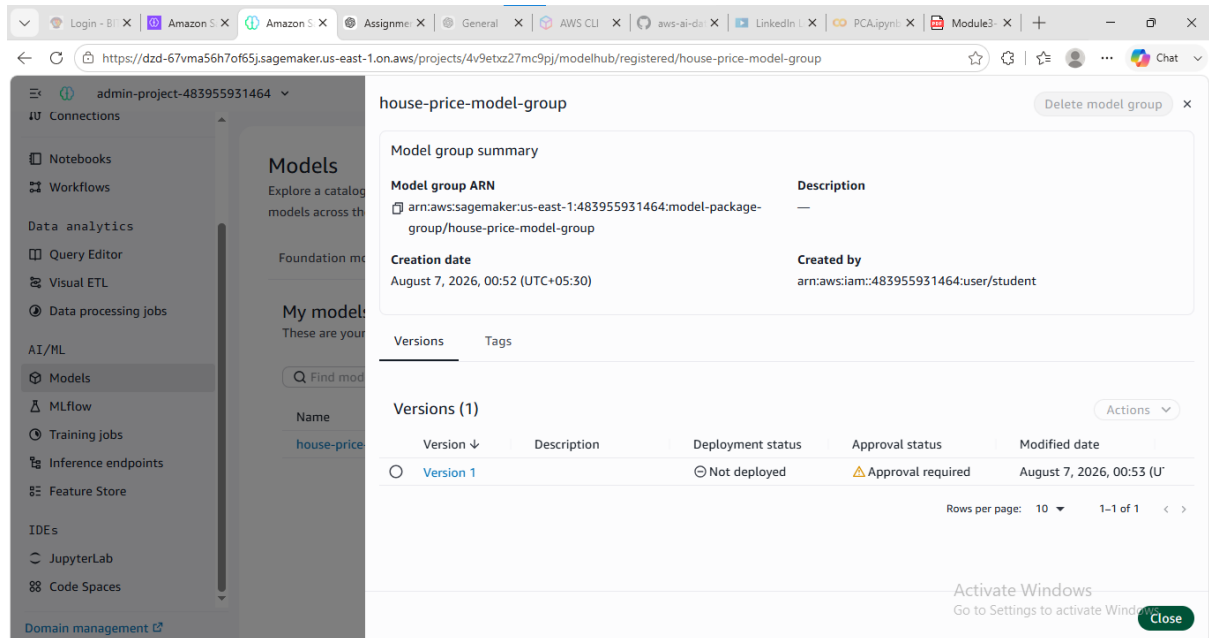


Step 9 – SageMaker Model Registry

Created a Model Package Group and successfully registered the trained model.

Model Status: PendingManualApproval

Fig 6 Model Registry:



6. Model Performance

The trained Random Forest model achieved the following performance.

Metric	Value
Mean Absolute Error (MAE)	26,019.20
Root Mean Squared Error (RMSE)	33,920.81
R ² Score	0.9131

Interpretation

- The model predicts house prices with an average error of approximately **\$26,000**.
- RMSE indicates that larger prediction errors are limited and the overall prediction quality is good.
- An R² score of **0.9131** indicates that the model explains approximately **91.31%** of the variation in house prices.

```
print("=" * 60)
print("Model Evaluation")
print("=" * 60)

mae = mean_absolute_error(y_test, predictions)
rmse = np.sqrt(mean_squared_error(y_test, predictions))
r2 = r2_score(y_test, predictions)

print(f"MAE : {mae:.2f}")
print(f"RMSE : {rmse:.2f}")
print(f"R² : {r2:.4f}")
```

[42] ✓ 0.0s Python

```
...
=====
Model Evaluation
=====
MAE : 26,019.20
RMSE : 33,920.81
R² : 0.9131
```

7. Feature Importance

The Random Forest algorithm identified the following feature importance.

Rank	Feature	Importance
1	size_sqft	0.638687
2	distance_km	0.272966
3	age_years	0.043274
4	bedrooms	0.036395
5	has_garage	0.008677

The results are consistent with the synthetic data generation formula, where **house size** contributes the most to the predicted price.

8. Results

The implementation was completed successfully.

The following AWS resources were created:

- Amazon S3 Bucket
- Glue Database
- Glue Table
- Athena Query
- SageMaker Feature Group
- Random Forest Model
- Model Artifact (model.tar.gz)
- Model Card (model_card.md)
- SageMaker Model Package Group

- SageMaker Model Package

All AWS resources were created successfully, the Random Forest model achieved an R^2 score of **0.9131**, and the trained model was successfully registered in SageMaker Model Registry with **PendingManualApproval** status.

9. Conclusion

This lab demonstrated the complete lifecycle of a machine learning workflow using AWS services.

The project successfully covered:

- Data generation
- Cloud storage using Amazon S3
- Metadata management using AWS Glue
- SQL querying using Amazon Athena
- Feature management using SageMaker Feature Store
- Local machine learning model training using Scikit-Learn
- Model packaging and registration using SageMaker Model Registry

The assignment provided practical experience in implementing an end-to-end MLOps workflow while keeping infrastructure costs minimal by avoiding SageMaker training instances.